TICKET BOOKING SOFTWARE REPORT

ResilientTicket

Self-Healing Railway Ticket Booking System

ResilientTicket is a railway ticket booking application designed to detect controlled software failures, automatically recover from them, and record the recovery process.

Core resilience cycle

01 Failure Injection

02 Detection

03 Diagnosis

04 Recovery

05 Service Restored

Technology stack

- React
- Node.js
- Express.js
- PostgreSQL
- REST APIs

Project objective

Build a railway booking system that can detect failures, automatically recover without human intervention, and provide measurable evidence of resilience.

SYSTEM ARCHITECTURE

Normal application flow

01 Passenger

02 React Frontend

03 Express.js REST API

04 PostgreSQL Database

Self-healing flow

01 Chaos Control

02 Failure Injection

03 Incident Detection

04 Diagnosis

05 Recovery Service

06 Database / Service Restoration

The system combines a normal railway booking workflow with a recovery layer that handles controlled failures. Each chaos test creates a specific failure, records an incident, waits for the configured recovery period, executes the appropriate recovery mechanism, and records the final outcome.

Incident lifecycle

DETECTED → DIAGNOSED → RECOVERING → RECOVERED

FAILURE INJECTION AND SELF-HEALING RESULTS

All four scenarios below were implemented and tested. Recovery times are measured/observed during project testing and should remain stated as approximately 30 seconds, not as more precise values.

Recovery target: < 30 seconds

The configured recovery mechanism is designed around a 30-second automatic recovery window.

MEASURED / OBSERVED

≈ 30 seconds

during project testing

Failure	How it is detected	How it heals	Measured recovery time
Stale / Orphaned Seat Lock	A chaos endpoint intentionally locks an eligible seat without an active pending or confirmed booking and records an ORPHANED_SEAT_LOCK incident.	After the configured recovery delay, the recovery process verifies that the seat is still safely recoverable, releases the orphaned lock, and marks the incident RECOVERED.	Approximately 30 seconds.
Ticket Generation Failure	A selected confirmed booking with a generated ticket is intentionally changed to FAILED ticket status and a TICKET_GENERATION_FAILURE incident is recorded.	The failed ticket recovery service regenerates the ticket and verifies that the booking returns to GENERATED status.	Approximately 30 seconds.
Train Search Service Failure	The train-search service is intentionally marked unavailable and a TRAIN_SEARCH_FAILURE incident is recorded.	The recovery service restores the train-search service state after the configured recovery delay.	Approximately 30 seconds.
Booking Service Failure	The booking service is intentionally marked unavailable and a BOOKING_SERVICE_FAILURE incident is recorded.	The recovery service restores the booking service state after the configured recovery delay.	Approximately 30 seconds.

BEFORE AND AFTER RESILIENCE

Before self-healing

- Failure can leavethe affected service or data state unresolved
- Manual intervention may be required
- No automatic incident lifecycle
- No automatic recovery measurement
- User-facing service disruption can persist

After ResilientTicket

- Controlled failures can be injected on demand
- Incidents are recorded
- Incident lifecycle is tracked
- Recovery is automatically triggered
- Recovery action is recorded
- Recovery time is measured
- System health can be monitored through the admin dashboard
- Recovery target is approximately 30 seconds

Resilience flow

01 **Failure**

02 **Detected**

03 **Diagnosed**

04 **Recovery Action**

05 **Recovered**

KNOWN LIMITATION — FAILURE THE SYSTEM CANNOT HEAL YET

ResilientTicket currently handles the four implemented and tested failure scenarios: stale/orphaned seat locks, ticket generation failure, train search service failure, and booking service failure. It does not provide automatic recovery for every possible infrastructure or application failure.

Complete backend process/server crash is not automatically healed by the current implementation. The recovery mechanisms run inside the backend process itself, so if the entire Node.js process stops, its in memory recovery state and timers also stop. An external process supervisor, container orchestrator, or infrastructure-level health manager would be required to restart the service automatically.

This limitation is intentional and is documented rather than hidden.

CONCLUSION

ResilientTicket demonstrates that a railway ticket booking application can be extended with controlled failure injection, incident tracking, automatic recovery, and measurable resilience. The project successfully demonstrates four self-healing failure scenarios and records their recovery lifecycle. The system also explicitly documents the boundary of its current recovery capabilities.

ANNEXURE: AI USE DECLARATION

AI TOOLS USED

AI Tool	How it was used
ChatGPT	Used for programming assistance, debugging, code explanations, architecture discussion, UI improvements, failure/recovery logic guidance, testing guidance, and documentation drafting.
Stitch AI	Used for UI/UX design exploration, interface layout ideas, visual design inspiration, and generating/refining frontend design concepts for the ResilientTicket application.
Gamma AI	Used to assist with the visual design, layout, formatting, and presentation of the Resilience Report PDF.

I confirm that this entry is our own work, that every AI tool used is listed above, and that I can explain every part of the submitted code.

Name : Lakshita Negi

Signature : 

Date : 27 - 10 - 2026